HEILBRONN UNIVERSITY
OF APPLIED SCIENCES

Deep Learning (285652)

Overfitting in Neural Networks

Wagner Nils* & Wenzel Nick†

April 1, 2026
Submitted to Carsten Lanquillon

*212405, nwagner1@stud.hs-heilbronn.de
†207678, nwenzel@stud.hs-heilbronn.de

Abstract

The present paper evaluates how effectively L1 regularization, L2 weight decay, Dropout, Batch Normalization, and Early Stopping control overfitting in multilayer perceptrons trained on MNIST. The experimental design varies both network depth and width and then applies regularization to a fixed reference architecture, with additional experiments on selected combinations. Across unregularized baselines, the generalization gap is already substantial in small models, ranging from 1.48 to 2.68 percentage points. Test accuracy improves with model size, but the gap remains persistent, indicating that higher capacity alone does not remove overfitting. Among single methods, Dropout at $p = 0.2$ yields the strongest test accuracy (98.23%) [2], while L2 with weight decay 10^{-2} yields the smallest gap (0.236 points) at the cost of lower accuracy, indicating underfitting [7]. Batch Normalization produces modest but consistent gains over matched baselines, whereas Early Stopping alone has limited effect in this setup. The strongest combined result is obtained by BatchNorm+Dropout on the 4×256 architecture, reaching 98.37% test accuracy with a 0.578-point gap. Regularization therefore becomes necessary already at modest network sizes, and the most effective solutions are balanced combinations rather than maximally strong single penalties.

1 Introduction

Overfitting in supervised learning occurs when a model achieves low empirical risk on the training sample while failing to achieve correspondingly low risk on unseen data [4, 5]. In neural networks, this phenomenon is often measured through the generalization gap, here operationalized as the difference between final training accuracy and best validation accuracy. A large gap indicates that the learned parameters fit the training distribution more closely than the broader data-generating process [1].

This issue is especially relevant for multilayer perceptrons (MLPs), whose representational capacity grows rapidly with width and depth. Although increased capacity can improve optimization and representation quality, it also enables memorization of idiosyncratic training patterns. Even for MNIST, a comparatively simple benchmark, high-capacity MLPs can approach perfect training accuracy while still leaving a measurable validation deficit.

Regularization methods address this problem through distinct mechanisms. L1 penalties encourage sparse parameter vectors, L2 weight decay discourages large weights [7], Dropout introduces stochastic feature suppression during training [2], Batch Normalization stabilizes intermediate activations and often improves optimization [3], and Early Stopping limits training duration once validation performance ceases to improve [1]. Because these mechanisms act differently, their comparative behavior and possible interactions merit direct empirical evaluation.

The research question examined here is: *How effectively do different regularization techniques prevent overfitting in MLPs, and at what network size does regularization become necessary?*

The comparison matters because practical model design depends not only on raw accuracy, but also on identifying when additional capacity stops yielding robust generalization and which interventions preserve performance most efficiently.

2 Methods

2.1 Dataset and preprocessing

The experiments use the MNIST handwritten digit dataset. The notebook indicates a split of 50,000 training samples, 10,000 validation samples, and 10,000 test samples. Inputs are converted to tensors and normalized using mean 0.1307 and standard deviation 0.3081, which is the standard normalization used for MNIST in PyTorch pipelines [1].

2.2 Model architecture

All models are fully connected MLPs operating on flattened $28 \times 28 = 784$-dimensional inputs and producing 10 output logits. The architecture family varies in two dimensions: depth, defined as the number of hidden layers, and width, defined as the hidden units per layer. Baseline depth experiments evaluate 1–5 hidden layers at width 64 and 256. A separate width experiment fixes depth at three hidden layers and varies width from 64 to 1024. Depending on the configuration, parameter counts range from 50,890 to 2,913,290.

2.3 Regularization techniques

L1 regularization is implemented through an additive penalty proportional to the sum of absolute weights [7]. This penalty was varied over $\lambda \in \{10^{-5}, 10^{-4}, 10^{-3}, 10^{-2}\}$. L2 regularization is implemented as optimizer weight decay [7] with values $\{10^{-5}, 10^{-4}, 10^{-3}, 10^{-2}\}$. Dropout randomly disables hidden activations during training [2] with rates $p \in \{0.1, 0.2, 0.3, 0.5\}$. Batch Normalization is applied to matched 2×256 and 4×256 architectures, enabling direct comparison against otherwise identical baselines [3]. Early Stopping monitors validation performance with patience values [1] 3, 5, and 10. In addition to isolated methods, four combined configurations are reported: L2+Dropout, BatchNorm+Dropout, L2+Dropout+Early Stopping, and a full combination of L2+BatchNorm+Dropout+Early Stopping.

2.4 Training protocol

The notebook trains most configurations for 30 epochs and uses 50 epochs for Early Stopping experiments, with termination occurring before the maximum epoch when patience is exhausted. The optimizer is Adam with learning rate 10^{-3}, and the batch size is 128. Performance is recorded as best validation accuracy, epoch of peak validation accuracy, final training accuracy,

test accuracy, and overfitting gap. Because the available results consist of aggregate outputs from single runs, no uncertainty estimates across random seeds are available; interpretation is therefore limited to the reported runs.

3 Results

3.1 Learning curves with and without regularization

Across all architectures, unregularized models exhibit a characteristic divergence between training and validation performance: training accuracy rapidly approaches near-perfect levels, while validation accuracy saturates early, producing a persistent generalization gap. This pattern reflects classical high-capacity overfitting behavior [1]. The gap is already non-trivial in small models (approximately 1.5–2.7 percentage points) and remains stable throughout training, indicating that additional epochs do not improve generalization.

Figure 1 illustrates this behavior for a representative 4×256 architecture. The divergence between training and validation accuracy becomes clearly visible after approximately 6 epochs in the unregularized model, when validation accuracy stops improving while training accuracy continues to increase. Across unregularized architectures, overfitting consistently emerges early, typically between epochs 5 and 8, whereas regularized models delay this divergence.

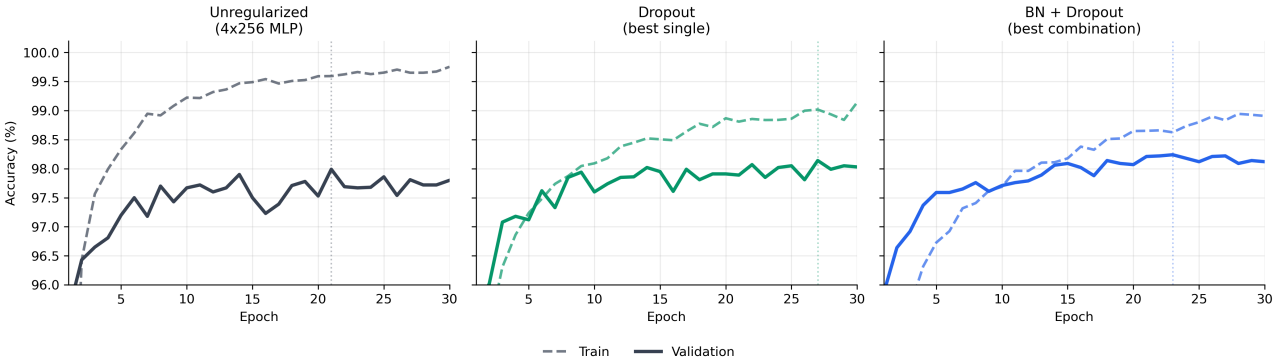

Figure 1: Training and validation accuracy over epochs for unregularized, Dropout, and BatchNorm + Dropout models (4×256 MLP).

In contrast, regularized models in Figure 1 display smoother and more stable convergence dynamics. Techniques such as L2 weight decay and Dropout reduce the slope difference between training and validation curves, effectively narrowing the generalization gap. Dropout in particular delays convergence while improving final validation performance, consistent with its stochastic regularization effect [2]. Early Stopping primarily truncates training before overfitting intensifies but does not fundamentally reshape the learning trajectory, confirming that it acts as a procedural rather than structural regularizer.

3.2 Comparison across techniques and network sizes

Increasing network size consistently improves test accuracy, but does not eliminate overfitting, indicating that capacity alone is insufficient for generalization. Among the evaluated methods, L2 regularization achieves the strongest reduction in generalization gap (down to approximately 0.24 percentage points at weight decay 10^{-2}), but at the cost of reduced test accuracy, suggesting underfitting at higher regularization strengths.

Dropout provides the most effective trade-off: at moderate rates ($p = 0.2$), it achieves the highest observed test accuracy (approximately 98.2–98.4%) while still reducing overfitting. This aligns with its ability to approximate model averaging [2]. Batch Normalization yields smaller but consistent improvements across architectures, stabilizing optimization and slightly improving generalization [3]. Early Stopping alone shows limited impact.

Importantly, the best-performing configurations emerge from combinations, particularly BatchNorm + Dropout in larger architectures (e.g., 4×256), indicating complementary effects between variance reduction and optimization stabilization. A clear trade-off emerges between minimizing the generalization gap and maximizing test accuracy: configurations with very small gaps, such as strong L2 or combined regularization with Early Stopping, often achieve lower test accuracy, whereas moderately regularized models, especially Dropout and BatchNorm + Dropout, provide a better balance between both objectives.

3.3 Overfitting threshold

The experiments demonstrate that the overfitting threshold is reached at surprisingly small model sizes. Even the smallest architecture (1×64) exhibits a generalization gap exceeding 2 percentage points, providing clear evidence that overfitting is not exclusively a large-model phenomenon. While increasing capacity slightly reduces the relative gap, it does not remove it, implying that the threshold is effectively crossed at minimal capacity.

Regularization shifts this threshold by mitigating gap growth: with appropriate techniques (especially Dropout or L2), larger models can be trained without proportional increases in overfitting. However, no configuration fully eliminates the gap, reinforcing the conclusion that regularization is necessary across all tested scales rather than only beyond a specific size boundary. Table 1 summarizes the most relevant configurations.

Method	Overfitting Gap (pp)	Test Accuracy (%)	Key Effect
Baseline	∼1.53	∼98.2	Strong overfitting
BatchNorm	∼1.48	∼98.3	Stabilizes training
L2 + Dropout	∼0.52	∼98.15	Strong gap reduction
BN + Dropout	∼0.58	∼98.35	Best overall trade-off
L2 + Dropout + ES	∼0.25	∼97.95	Minimal gap, slight underfit

Table 1: Comparison of regularization techniques on a 4×256 MLP.

4 Discussion

The results show that regularization is already warranted at comparatively small network sizes. Even 1×64 and 3×64 models exhibit gaps above two percentage points, and no unregularized configuration removes the discrepancy between training and validation performance. Larger models improve test accuracy, but their gains arise alongside continued near-saturation of training accuracy, so increased capacity alone is not an adequate defense against overfitting [1].

Among individual techniques, Dropout provides the strongest overall balance between generalization and accuracy [2]. In particular, $p = 0.2$ improves test accuracy relative to the matched 2×256 baseline while markedly reducing the gap. This can be explained by its ability to reduce co-adaptation between neurons and implicitly approximate an ensemble of subnetworks [2]. L2 is the most effective direct mechanism for shrinking the gap, but its strongest setting does so by sacrificing too much predictive performance [7]. L1 is even more sensitive: small penalties are viable, but large penalties quickly become destructive. Batch Normalization behaves differently from explicit penalties [3]. Its gains are modest in isolation, yet consistently positive, and its strongest contribution appears when paired with Dropout in the combined experiments [6].

A non-trivial finding is that the configuration with the smallest overfitting gap is not the one with the best test accuracy. L2+Dropout+Early Stopping reaches a 0.252-point gap, but BatchNorm+Dropout attains materially higher test accuracy at 98.37% with only a slightly larger gap of 0.578. This indicates that minimizing the gap alone is not equivalent to maximizing generalization quality; beyond a certain point, stronger regularization pushes the model toward underfitting. The negative gaps observed for very strong L1 and high-dropout settings reinforce this interpretation.

Regarding interaction effects, the results indicate that combining techniques can yield substantial improvements, but not all combinations are equally beneficial. In particular, the BatchNorm + Dropout pairing consistently outperforms individual methods, suggesting that their effects are complementary rather than redundant. However, more aggressive combinations (e.g., L2 + Dropout + Early Stopping) may over-regularize the model and reduce accuracy.

The paper has several limitations. All results are based on MNIST, which is considerably

simpler than modern large-scale image benchmarks, and all evaluated architectures are MLPs rather than convolutional networks. In addition, repeated runs of the same configuration exhibited slight variations in the reported metrics. These fluctuations are most likely caused by the stochastic nature of the training process, in particular the random shuffling of the training data (`shuffle=True` in the DataLoader), which leads to different data orders across runs and consequently to slightly different optimization trajectories. As a result, small performance differences between methods should be interpreted with caution, as they may partly reflect this inherent randomness rather than systematic effects. At the current stage, no definitive fix or control mechanism for this source of variability has been implemented. The conclusions should therefore be understood as valid within the reported experimental regime rather than as universally stable rankings of the methods.

References

[1] *Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep Learning. MIT Press* https://www.deeplearningbook.org/ abgerufen am 30.03.2026

[2] Srivastava, N. et al. (2014). Dropout: A Simple Way to Prevent Neural Networks from Overfitting. JMLR, 15.

[3] Ioffe, S., & Szegedy, C. (2015). Batch Normalization. ICML.

[4] Bisong, E. (2019). Machine Learning on Google Cloud Platform. Apress.

[5] Rajvanshi, S. et al. (2024). Overfitting in Machine Learning. Springer.

[6] Tian, Y., & Zhang, Y. (2022). Regularization Strategies in Machine Learning. Information Fusion.

[7] Krogh, A., & Hertz, J. A. (1992). A Simple Weight Decay Can Improve Generalization. NeurIPS.